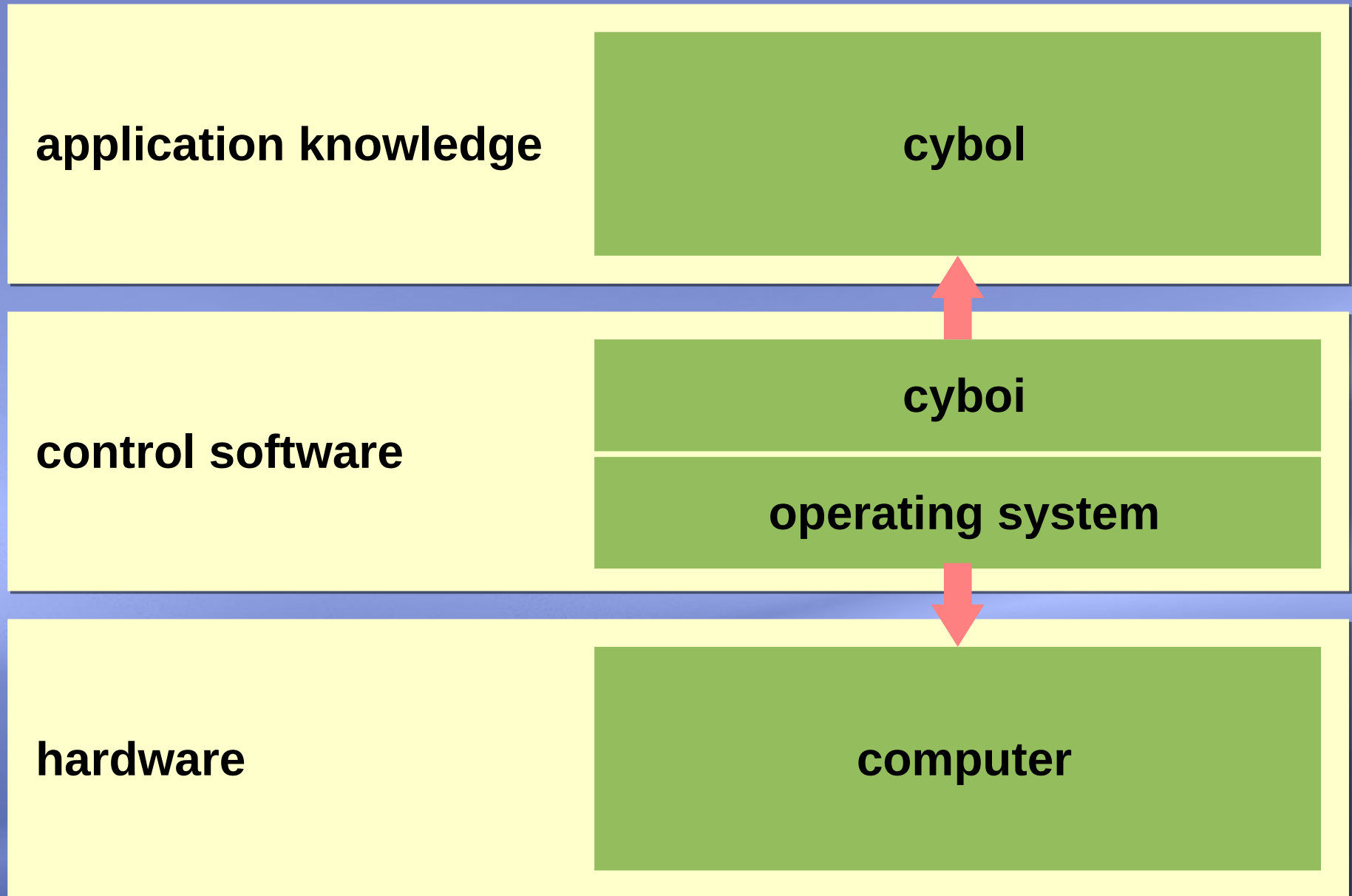


Cybernetics Oriented Programming (CYBOP)



Christian Heller

<christian.heller@tuxtax.de>



cyboi app/run.cybol

criterion	java world	cybop world
language	java	cybol (xml-based)
interpreter	java virtual machine (jvm)	cyboi (written in c)
theory	object oriented programming (oop)	cybop (knowledge schema)

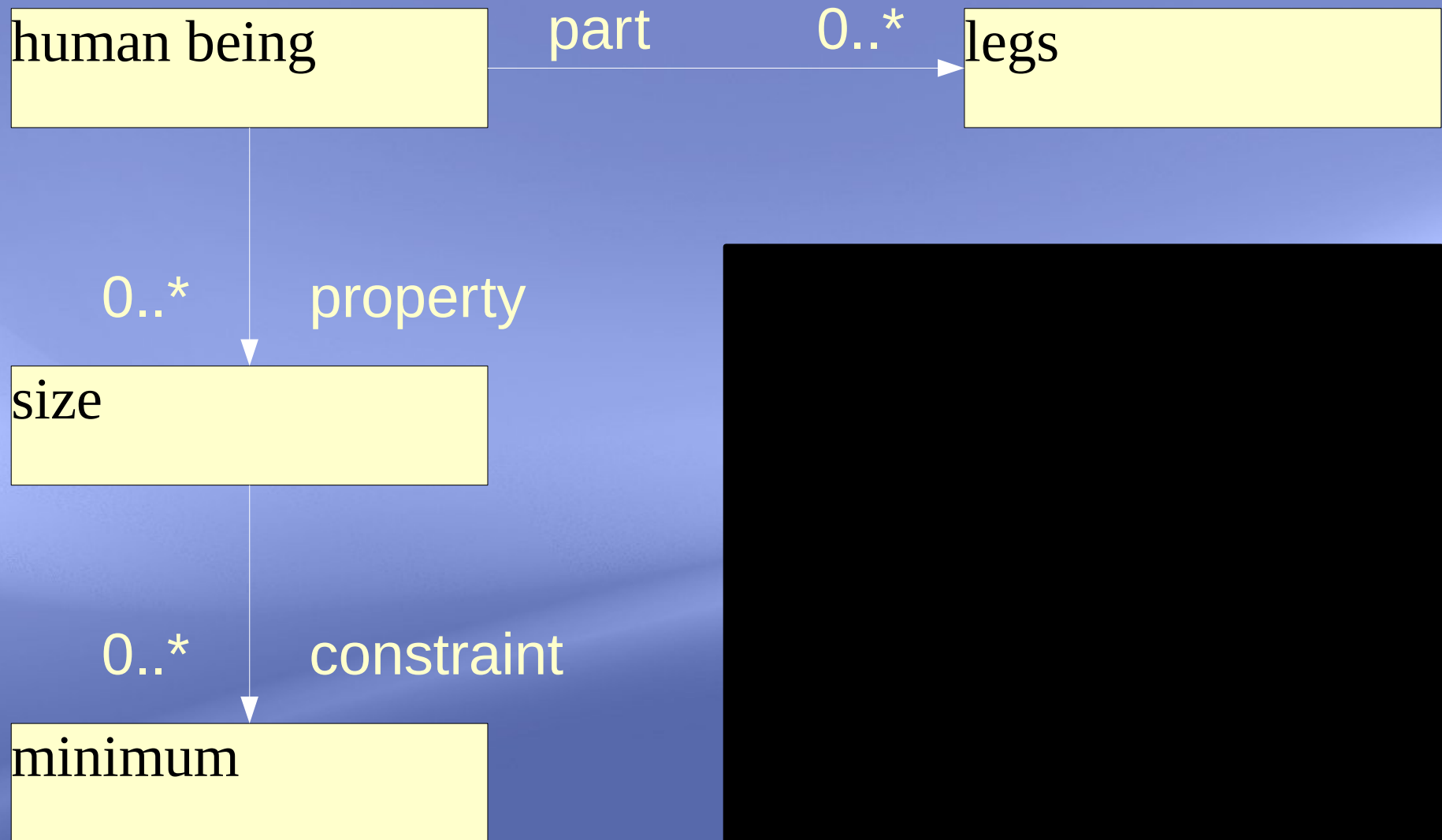
java class

```
public class Horse extends java.lang.Object {
```

```
    private String name = "Fury";  
    private Body body = new Body();  
    private Rectangle size = new Rectangle();  
    private String colour = "black";  
    private String mood = "happy";
```

```
    public void ride(String command, int speed);  
    public void feed(String food, String dung);
```

```
<constraint name="colour" channel="inline" abstraction="choice" model="black,white"/>  
<part name="mood" channel="inline" abstraction="set" model="happy,sad,aggressive"/>
```



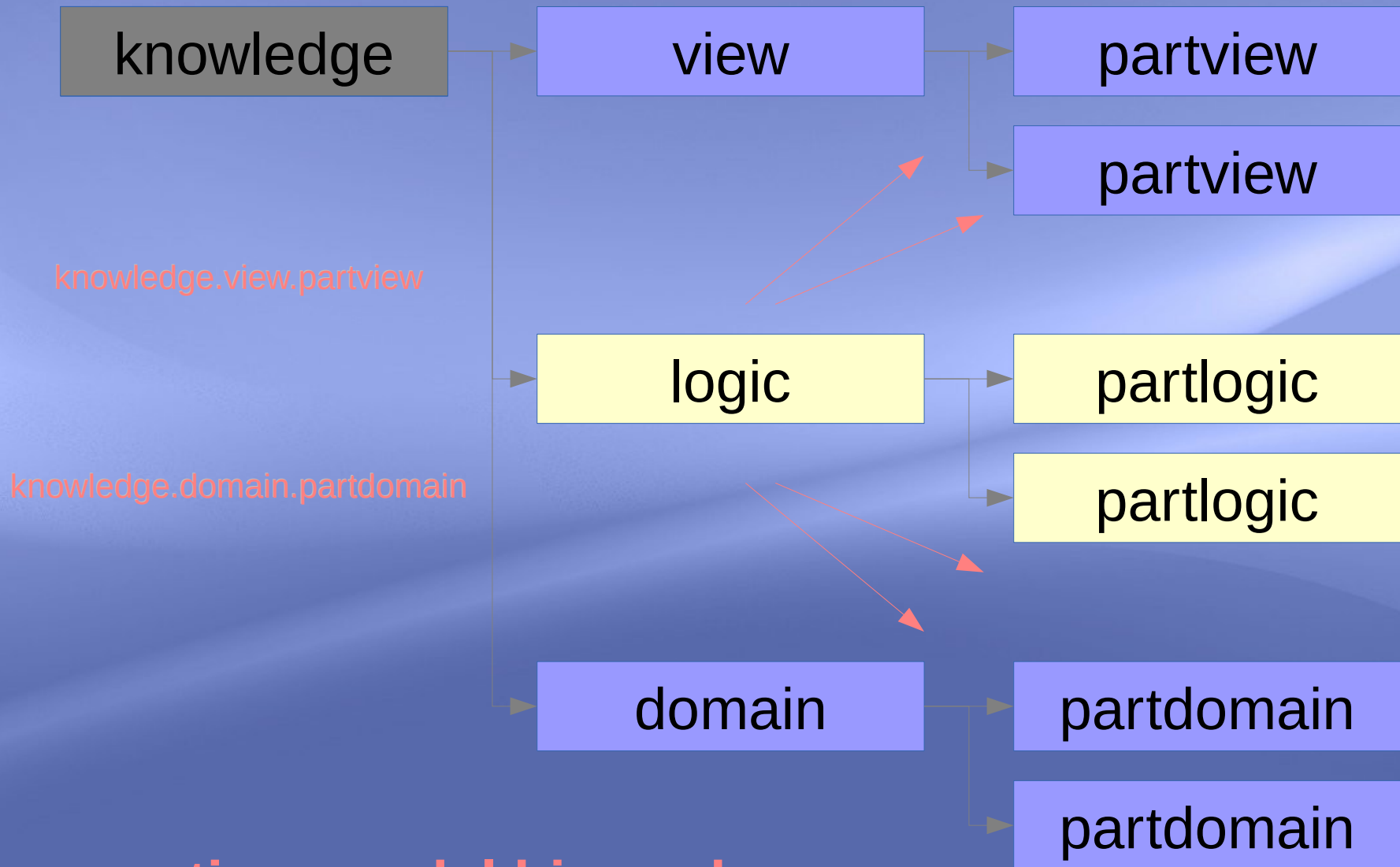
double hierarchy – in space, time etc.

cybol state template

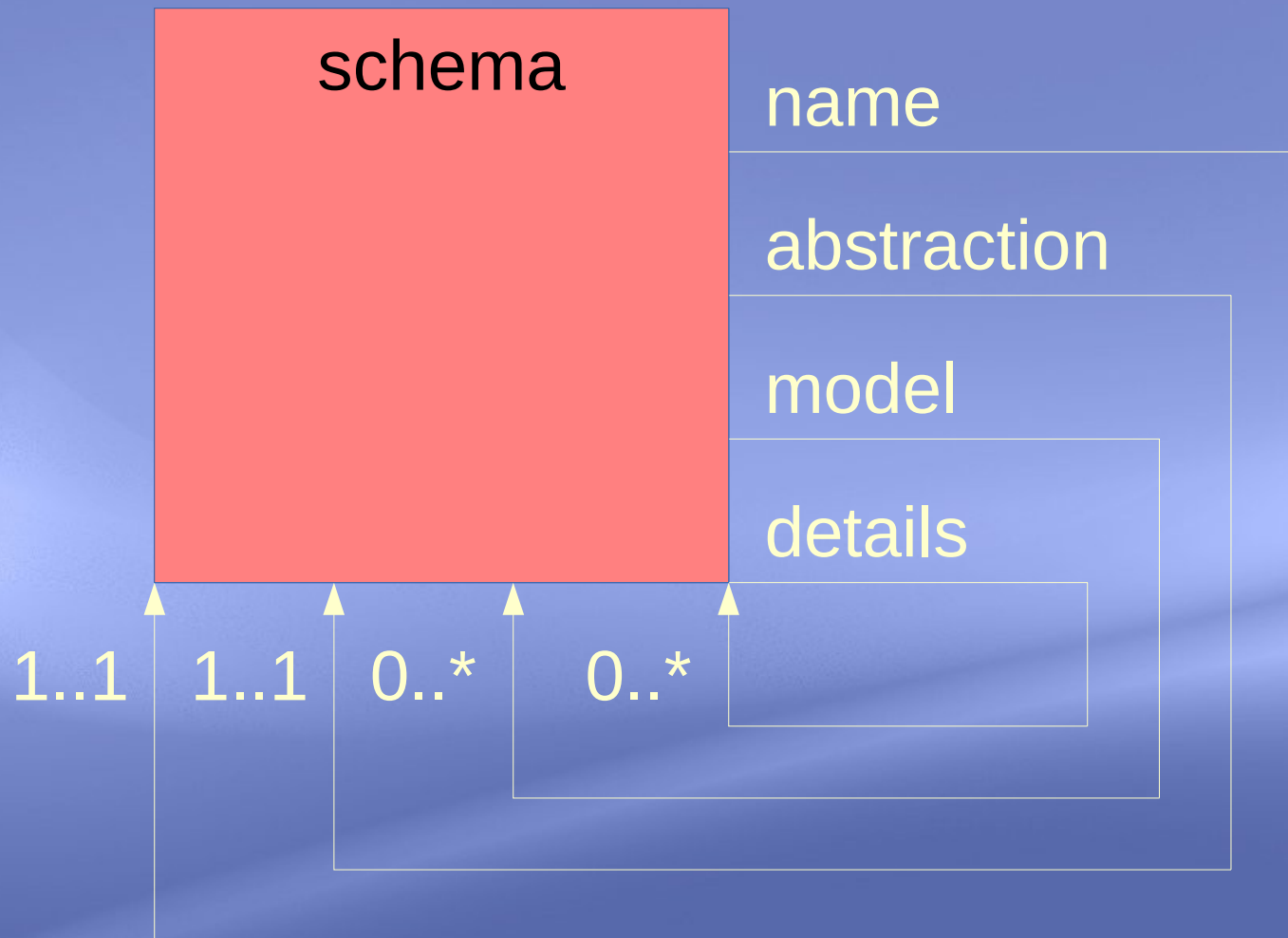
```
<model>
  <part name="name" channel="inline" abstraction="string" model="Fury"/>
  <part name="body" channel="file" abstraction="cybol" model="body.cybol">
    <property name="size" channel="inline" abstraction="vector" model="3, 1, 2"/>
    <property name="colour" channel="inline" abstraction="string" model="black">
      <constraint name="colour" channel="inline" abstraction="choice"
model="black,white"/>
    </property>
  </part>
  <part name="mood" channel="inline" abstraction="set" model="happy,sad,aggressive"/>
</model>
```

cybol logic template

```
<model>
  <part name="ride" channel="inline" abstraction="operation" model="ride">
    <property name="command" channel="inline" abstraction="string" model="galopp"/>
    <property name="speed" channel="inline" abstraction="knowledge"
model="horse.speed"/>
  </part>
  <part name="feed" channel="inline" abstraction="operation" model="feed">
    <property name="food" channel="inline" abstraction="string"
model="feeding.hay"/>
    <property name="apples" channel="inline" abstraction="knowledge"
model="feeding.apples"/>
  </part>
</model>
```



**runtime model hierarchy
with logic accessing states**

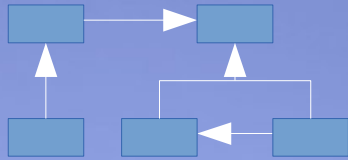


item
+ category
+ compound
= schema

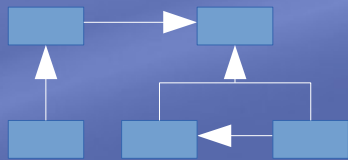
abstraction

relations
unidirectional

traditional

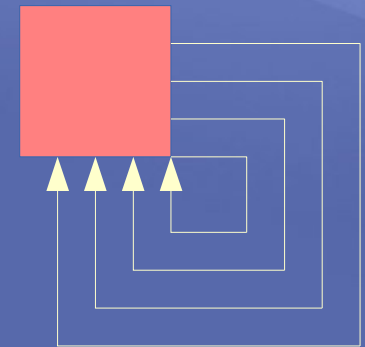
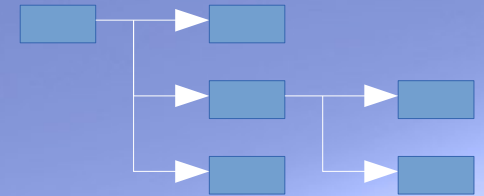


programme structure



runtime structure

cybop





traditional



cybop

models suffer from complexity

strong coupling / dependencies

inflexible

difficult to maintain

→ one schema as memory structure

→ directed acyclic graph (tree)

→ easily extensible

→ long-life software system

declarative language cybol
programming theory cybop
interpreter cyboi

flexible and adaptable at runtime
easy transparent communication

prototype-based programming similar to JavaScript

application creation by domain experts without programming knowledge

identical syntax for state (structure) and logic (function)
uniform usage of elements/commands

oriented on human thinking (which OOP only partly is)
double hierarchy separating micro-/macrocosm structure from meta properties
name „cybernetics”, because many of its original ideas were taken from nature

all knowledge hierarchical in one application tree
comparable to representation of website as DOM tree

cybol: just 4 tags and 4 attributes
unnecessary data are just ignored without error

bundling of attributes and methods – coupling, no flexibility

reflective meta architectures – bidirectional dependencies

bidirectional dependencies – complexity, circular references

global / static data access – untraceable data manipulation

container inheritance – falsified container content

problems

characteristics

long-life software system due to strict separation of application knowledge and system software
less dependencies due to representation of structural- and meta data in form of a double hierarchy
flexible architecture due to unified schema as the only remaining static memory structure
clean knowledge model code due to tree structure with unidirectional relations
universal communication due to star-like (not layer-like) translation architecture
easy handling due to one syntax describing states (space) and algorithms (time) in the same way
superfluous implementation phase due to direct interpretation of knowledge

solved problems

falsified container content due to container inheritance
fragile base class problem due to polymorphism
untraceable data manipulation due to global data access
circular references due to bidirectional dependencies
bidirectional dependencies due to reflective meta architectures
strong coupling due to bundling of attributes and methods
difficult maintenance due to high complexity



although cybop is free / open source software (GPL), it may well be used for commercial applications, since enterprise knowledge remains private

cybop is not in research for very much longer – discover its potential !

<http://www.cybop.org/>

Cybernetics Oriented Programming (CYBOP)



Christian Heller

<christian.heller@tuxtax.de>

<http://www.cybop.org/>

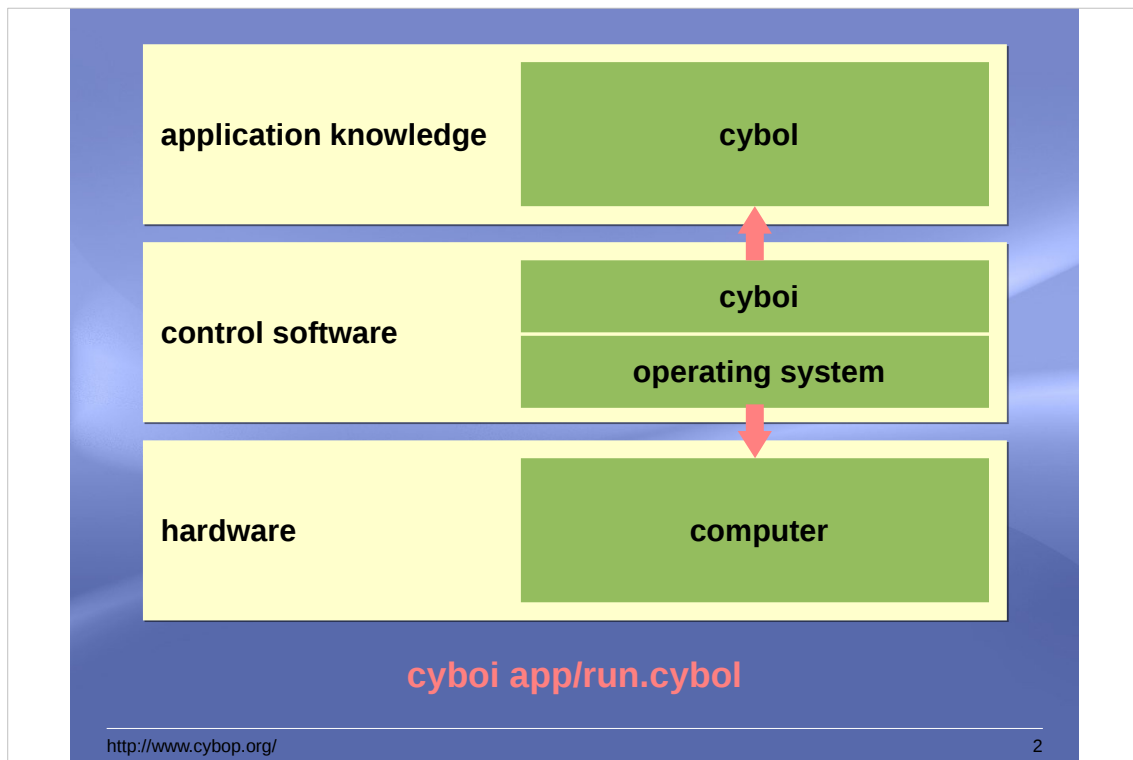
1

Liebe Gleichgesinnte,

ich möchte heute ein freies Softwareprojekt namens CYBOP vorstellen, an welchem ich seit nunmehr 14 Jahren arbeite.

Den Begriff „Kybernetik-orientiert“ habe ich gewählt, da viele Ideen Informatik-fremder Fachbereiche in das Projekt eingeflossen sind.

Beispiele nenne ich am Ende des Vortrages.



Das Projekt besteht aus:

- der XML-basierten Sprache CYBOL
- dem in C geschriebenen Interpreter CYBOI

Der Programmstart erfolgt wie bei jeder anderen interpretierten Sprache.

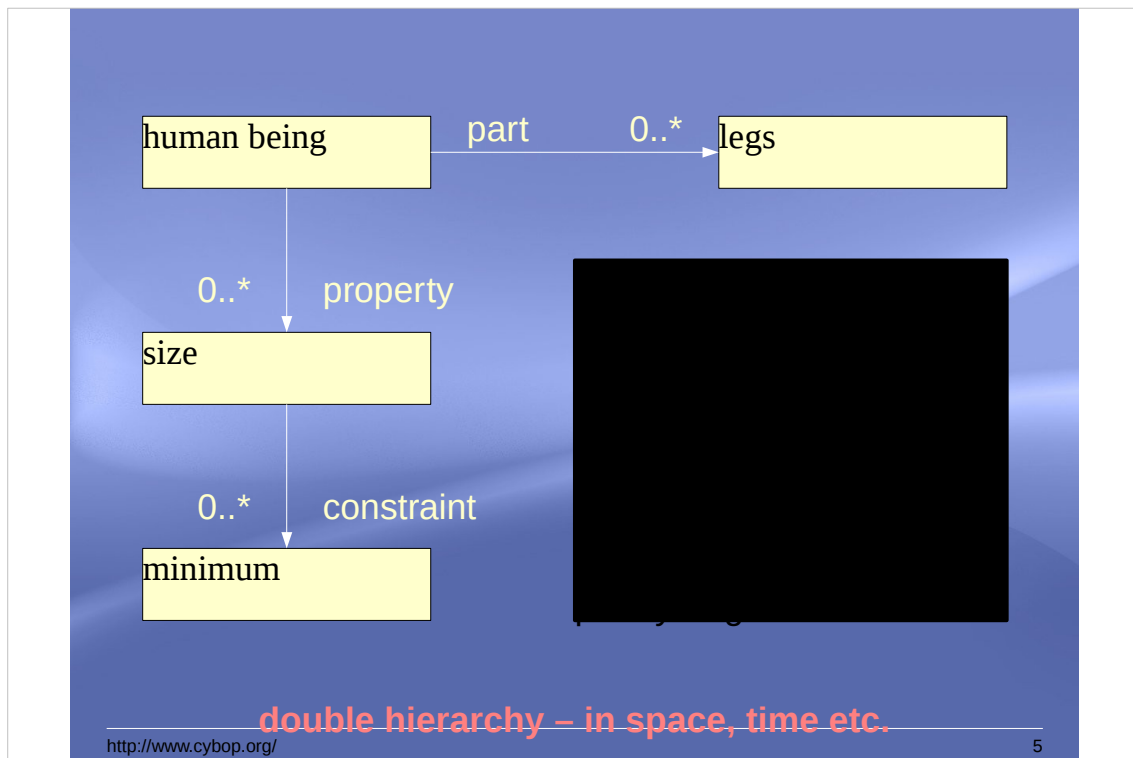
critterion	java world	cybop world
language	java	cybol (xml-based)
interpreter	java virtual machine (jvm)	cyboi (written in c)
theory	object oriented programming (oop)	cybop (knowledge schema)

<http://www.cybop.org/> 3

Beispielhaft vergleicht diese Folie die Welt der Programmiersprache Java mit CYBOP.

java class

```
public class Horse extends java.lang.Object {  
    private String name = "Fury";  
    private Body body = new Body();  
    private Rectangle size = new Rectangle();  
    private String colour = "black";  
    private String mood = "happy";  
  
    public void ride(String command, int speed);  
    public void feed(String food, String dung);  
  
<constraint name="colour" channel="inline" abstraction="choice" model="black,white"/>  
<part name="mood" channel="inline" abstraction="set" model="happy,sad,aggressive"/>  
}
```



Die Vermischung von Strukturinformationen (im Sinne des Mikro-/Makrokosmos) und zusätzlichen Meta-Eigenschaften in quasi allen herkömmlichen Programmiersprachen ist eine Tragödie.

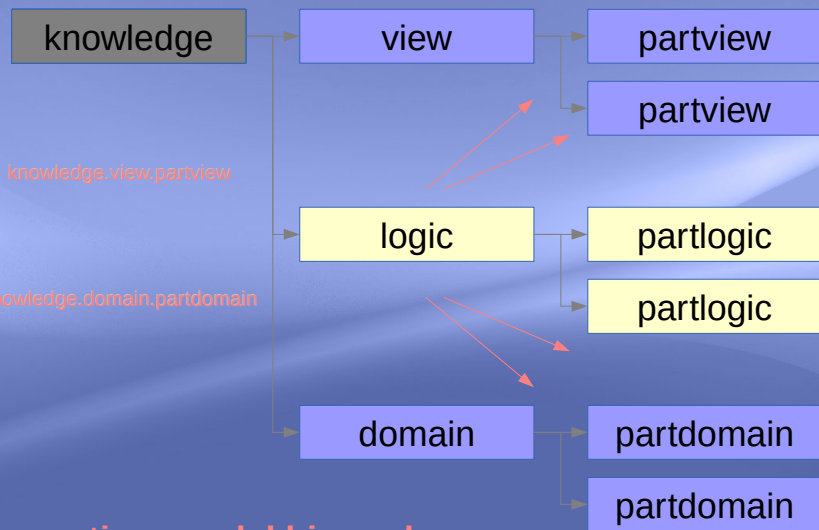
Erst ihre Unterscheidung mittels Doppelhierarchie macht es möglich, Anwendungsmodelle uniform zu verarbeiten.

cybol state template

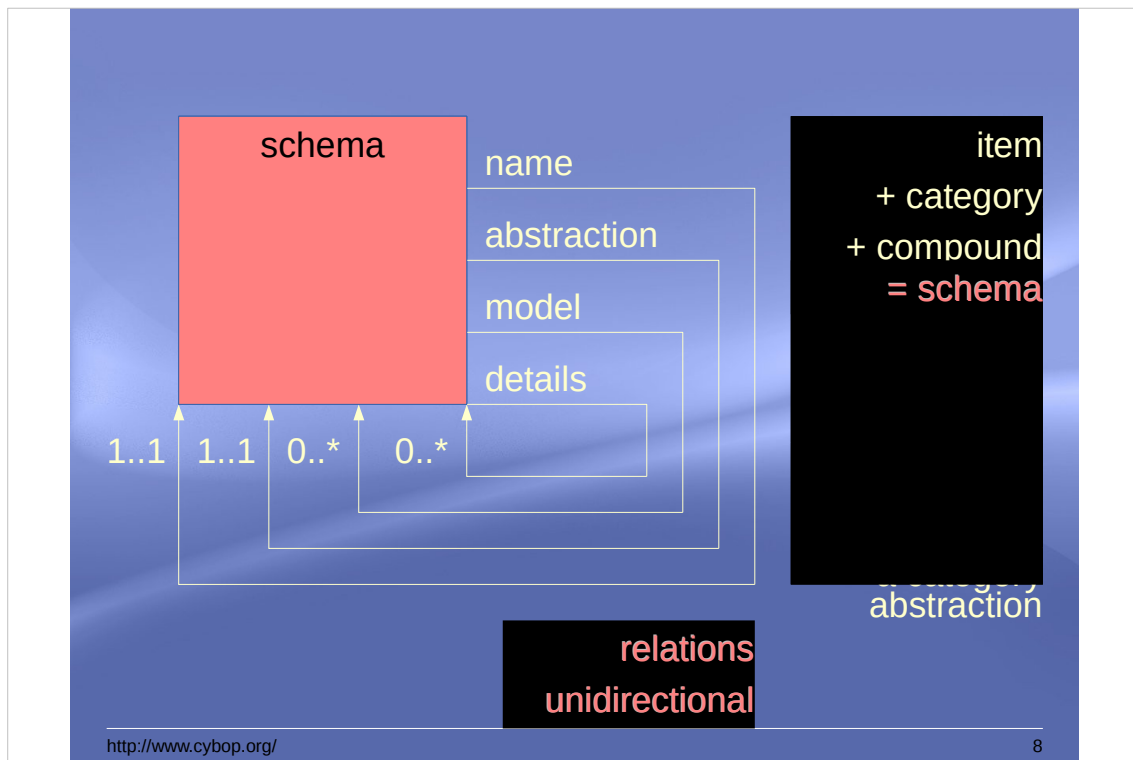
```
<model>
  <part name="name" channel="inline" abstraction="string" model="Fury"/>
  <part name="body" channel="file" abstraction="cybol" model="body.cybol">
    <property name="size" channel="inline" abstraction="vector" model="3, 1, 2"/>
    <property name="colour" channel="inline" abstraction="string" model="black">
      <constraint name="colour" channel="inline" abstraction="choice"
model="black,white"/>
    </property>
  </part>
  <part name="mood" channel="inline" abstraction="set" model="happy,sad,aggressive"/>
</model>
```

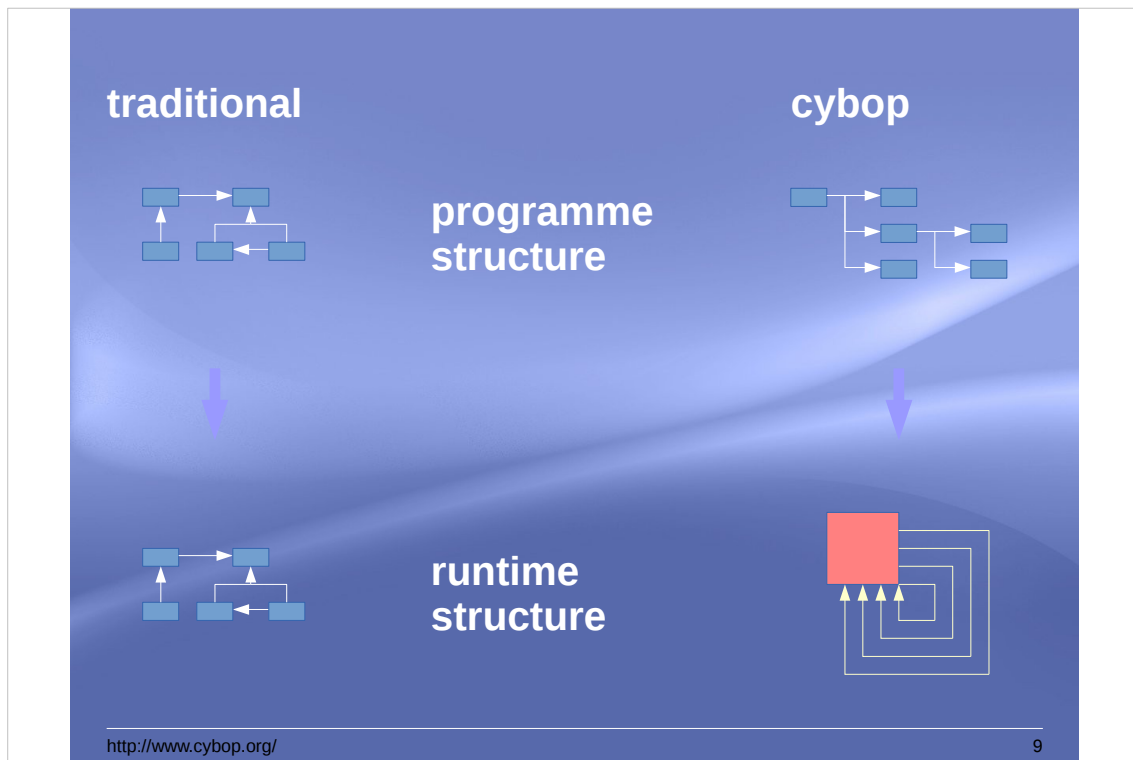
cybol logic template

```
<model>
  <part name="ride" channel="inline" abstraction="operation" model="ride">
    <property name="command" channel="inline" abstraction="string" model="galopp"/>
    <property name="speed" channel="inline" abstraction="knowledge"
model="horse.speed"/>
  </part>
  <part name="feed" channel="inline" abstraction="operation" model="feed">
    <property name="food" channel="inline" abstraction="string"
model="feeding.hay"/>
    <property name="apples" channel="inline" abstraction="knowledge"
model="feeding.apples"/>
  </part>
</model>
```



runtime model hierarchy
with logic accessing states





Universelles Schema als Speicherstruktur

Flexibilität statt starrer Strukturen



traditional



cybop

models suffer from complexity

strong coupling / dependencies

inflexible

difficult to maintain

→ one schema as memory structure

→ directed acyclic graph (tree)

→ easily extensible

→ long-life software system

declarative language cybol
programming theory cybop
interpreter cyboi

flexible and adaptable at runtime
easy transparent communication

prototype-based programming similar to JavaScript

application creation by domain experts without programming knowledge

identical syntax for state (structure) and logic (function)
uniform usage of elements/commands

oriented on human thinking (which OOP only partly is)
double hierarchy separating micro-/macrocosm structure from meta properties
name „cybernetics“, because many of its original ideas were taken from nature

all knowledge hierarchical in one application tree
comparable to representation of website as DOM tree

cybol: just 4 tags and 4 attributes
unnecessary data are just ignored without error

bundling of attributes and methods – coupling, no flexibility

reflective meta architectures – bidirectional dependencies

bidirectional dependencies – complexity, circular references

global / static data access – untraceable data manipulation

container inheritance – falsified container content

problems

characteristics

long-life software system due to strict separation of application knowledge and system software
less dependencies due to representation of structural- and meta data in form of a double hierarchy
flexible architecture due to unified schema as the only remaining static memory structure
clean knowledge model code due to tree structure with unidirectional relations
universal communication due to star-like (not layer-like) translation architecture
easy handling due to one syntax describing states (space) and algorithms (time) in the same way
superfluous implementation phase due to direct interpretation of knowledge

solved problems

falsified container content due to container inheritance
fragile base class problem due to polymorphism
untraceable data manipulation due to global data access
circular references due to bidirectional dependencies
bidirectional dependencies due to reflective meta architectures
strong coupling due to bundling of attributes and methods
difficult maintenance due to high complexity



although cybop is free / open source software (GPL), it may well be used for commercial applications, since enterprise knowledge remains private

cybop is not in research for very much longer – discover its potential !

<http://www.cybop.org/>

<http://www.cybop.org/>

13

Ich würde mich sehr freuen, interessierte Mitstreiter zu finden.